

PHASE 5 — SHARPENING THE BLADE

Explainer Video Scripts

*Advanced thinking patterns · data mastery · quality reflection
21 scripts · single protagonist “Ali” · house scripting standard*

Each beat is one spoken sentence. Hand each script to the review gate
(reviewing-explainer-scripts) before any art, voice, or render.

Phase 5 — "Sharpening the Blade" · Explainer Video Scripts

Advanced thinking patterns, data mastery, and quality reflection. Every script follows the house scripting standard: single protagonist ****Ali**** (an engineer at an ed-tech company learning to orchestrate AI coding agents), one concept per video, pyramid lead, one spoken sentence per beat, ≥2 emotional checkpoints, no title cards / no file-homework, closing spoken reflection. Hand each to `reviewing-explainer-scripts` before art/TTS.

Coverage map — session teaching points → script

Session teaching point	Covered in
Consumer vs producer mindset	SC-1, SLO 5.4
Autonomy — agent owns tasks, diagnoses/fixes its own errors	SC-1
Autonomous systems fail repeatedly (expected, not failure)	SC-1, SLO 5.1b
Skills as reusable building blocks (avoid re-explaining)	SLO 5.5, SC-1, SC-3
Self-improvement — feed corrections back into skills/harness	SLO 5.5, SC-1
Why evals — "how does the model know what it built is correct?"	SLO 5.1b
Plan first (documented plan in a file, actually read it)	SC-2
Implement, then verify	SC-2
Match plan vs. implementation as verification	SC-2
Automate browser testing via Chrome DevTools MCP	SC-3
Layered eval types (unit tests · hooks · browser E2E · output scoring)	SC-3, SLO 5.1b, SLO 5.7
Hooks intercept risky actions (e.g. DB delete → ask permission)	SC-3
Evaluating AI-generated outputs (scoring / classifying)	SLO 5.7, SLO 5.1b
Test cases in BDD format (specific Given/When/Then)	SC-4, SLO 5.1b
Organize test cases under docs/features/..., review before running	SC-4
Generate test cases from the codebase if no plan exists	SC-4
Turn the workflow into a reusable skill (e2e-chrome-testing)	SLO 5.5, SC-3
Slash commands to trigger skills reliably (/plan)	SC-2, SLO 5.5
Generalizes to mobile / Figma; backend validates functions directly	SC-3
Limits of "give the agent tools and let it self-verify"	SC-3 (close)

Section A — Session-Core Workflow Scripts

These four cover the session's central narrative (consumer→producer, the plan→verify workflow, browser self-testing, and BDD test cases) that the granular SLOs don't fully hit.

SC-1 — From consumer to producer (autonomy) [compulsory]

Concept: Shift from a consumer (tell the AI what to do, fix things yourself) to a producer (build a system that runs itself), because autonomy means the agent owns the task — even diagnosing and fixing its own errors.

Ali's scenario: Ali copy-pastes every error back to his agent by hand, then builds a system where the agent catches and fixes its own errors.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Stop telling the agent what to do; build a system that runs itself.</i>	Consumer to producer	Ali stepping back from the keyboard as a system hums on its own
2	scene	<i>Ali used his agent like a vending machine: ask, receive, repeat.</i>	The consumer habit	Ali pressing buttons on an "AI vending machine"
3	ali	<i>Every time it errored, he copied the message back by hand.</i>	Copy-paste the error	Ali pasting a red error into the chat [overlay] error text moving into a prompt
4	scene	<i>That hands-on habit feels responsible, and nearly everyone starts there.</i>	We all start here	warm scene, Ali busy but smiling
5	info	<i>A consumer directs the tool; a producer builds the system.</i>	Two mindsets	[info] two-column compare — Consumer "tell it, fix it yourself" vs Producer "build it, it runs itself"
6	ali	<i>Ali was the bottleneck every single time it stumbled.</i>	You are the bottleneck	Ali as a narrow gate that every task waits behind
7	info	<i>The producer's goal is autonomy: less human dependency.</i>	Autonomy	[info] definition card — "Autonomy = the agent owns the task"
8	ali	<i>So Ali let the agent read its own errors and try a fix.</i>	It fixes itself	the agent catching its own red error and patching it [overlay] "error → self-fix"
9	scene	<i>Handing over control feels risky, and that unease is normal.</i>	The unease is normal	Ali watching, hands off, slightly tense
10	info	<i>Autonomy is a dial you turn slowly, not a switch you flip.</i>	Turn the dial slowly	[info] a dial sliding from "manual" toward "autonomous"
11	info	<i>Expect it to fail as it does more; that is the process.</i>	Failing is expected	[info] a rising "autonomy" line with small dips labeled "normal"
12	ali	<i>He gave it tools to diagnose, retry, and verify on its own.</i>	Give it the tools	the agent with a toolkit working independently
13	ali	<i>Now Ali reviews outcomes instead of babysitting every step.</i>	Review, don't babysit	Ali relaxed, checking results on a dashboard
14	info	<i>Producer mindset trades doing the work for owning the system.</i>	Own the system	[info] before/after — "does tasks" vs "runs a system"
15	ali	<i>Same engineer, far more shipped, far less copy-pasting.</i>	More shipped, less grind	Ali calm as many features flow out of a pipeline
16	ali	<i>Where are you still the bottleneck your system could remove?</i>	Your turn	Ali turning gently to face the viewer

SC-2 — Plan first, then verify [compulsory]

Concept: Always write a documented plan in a file before building and actually read it, then implement, verify, and match the build against the plan.

Ali's scenario: Ali dives straight into building, gets the wrong thing, then writes a plan file first, reads it, builds, and checks the result against it.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Write the plan in a file before you build anything.</i>	Plan first	Ali opening a fresh plan.md before

				touching code
2	scene	<i>Ali used to just tell his agent 'build it' and hope.</i>	Straight to building	Ali and the agent rushing into a tangle of code
3	ali	<i>The agent built fast, but it built the wrong thing.</i>	Fast, but wrong	a finished feature that clearly misses the mark
4	scene	<i>Skipping the plan feels faster, and we have all done it.</i>	We all skip it	warm scene, Ali looking a little sheepish
5	info	<i>A plan is a written file you make before implementation.</i>	What is a plan?	[info] definition card — "Plan = documented intent, in a file"
6	ali	<i>So Ali had the agent write the plan first, then he read it.</i>	Read the plan	Ali actually reading the plan, pen in hand [overlay] plan.md open on screen
7	ali	<i>Reading it, he caught a wrong assumption before any code existed.</i>	Catch it early	Ali circling a flawed line in the plan
8	info	<i>The order is simple: plan, implement, then verify.</i>	Plan, build, verify	[info] a 3-step flow that builds one step at a time
9	scene	<i>Slowing down to plan feels backwards, yet it saves hours.</i>	Slow is fast	warm scene, Ali calm beside an hourglass
10	ali	<i>He triggered the plan with a slash command, so it never got skipped.</i>	Trigger it reliably	Ali typing <code>"/plan"</code> and a plan template appearing [overlay] <code>"/plan"</code>
11	ali	<i>After building, Ali compared the result against the plan.</i>	Match plan vs build	Ali holding the plan beside the built feature, checking
12	info	<i>Plan-versus-implementation is itself a form of verification.</i>	Compare = verify	[info] two panels — "planned" vs "built" with checkmarks
13	ali	<i>Two items were missing, so he sent the agent back for them.</i>	Fill the gaps	Ali marking two unmet items; the agent resumes
14	ali	<i>Only when build matched plan did Ali call it done.</i>	Done means matched	plan and build aligning under a green check
15	info	<i>A plan you skip is a plan you cannot verify against.</i>	No plan, no check	[info] contrast — a vague idea vs a clear, checkable plan
16	ali	<i>Before your next build, what would your plan file say?</i>	Your turn	Ali turning to face the viewer

SC-3 — Let the agent test itself in the browser [compulsory]

Concept: Connect Chrome DevTools to the coding agent (via MCP) so it opens the running app, catches errors, fixes them, and re-verifies — automating manual browser testing, with safety hooks for risky actions.

Ali's scenario: Ali manually clicks through his app after every change, then wires up Chrome DevTools MCP so the agent tests itself.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Give your agent a browser so it can test the app itself.</i>	Let it self-test	the agent driving a browser while Ali watches
2	scene	<i>After every change, Ali clicked through the app by hand.</i>	Manual clicking	Ali tiredly clicking buttons across screens

3	ali	<i>It was slow, and he still missed bugs users later hit.</i>	Slow and leaky	a bug slipping past Ali onto a user's screen
4	scene	<i>Manual testing drains everyone; you are not alone in dreading it.</i>	We all dread it	warm scene, Ali sighing at a long checklist
5	info	<i>MCP is a bridge that lets the agent use real tools, like Chrome.</i>	What is MCP?	[info] definition card — "MCP = agent-to-tools bridge"
6	ali	<i>Ali connected Chrome DevTools to his coding agent.</i>	Connect the browser	a cable linking the agent to a Chrome window [overlay] "Chrome DevTools MCP"
7	ali	<i>Now the agent opens the local app and actually looks at it.</i>	It opens the app	the agent viewing the running app in a browser
8	ali	<i>It reads console errors the way Ali used to, but instantly.</i>	It reads the errors	the agent scanning a red console line
9	info	<i>The loop: open, check, catch an error, fix, re-verify.</i>	The self-test loop	[info] a 5-step cycle diagram that animates around
10	scene	<i>Trusting the agent to verify feels strange at first, and that is fine.</i>	Strange but safe	warm scene, Ali hands off, calm
11	info	<i>This is one layer: functional, end-to-end browser testing.</i>	One eval layer	[info] a layer stack — unit tests
12	ali	<i>A safety hook still pauses it before anything truly risky.</i>	Hooks guard risk	the agent stopped at a "delete database?" gate, asking Ali [overlay] "permission needed"
13	ali	<i>The agent caught a broken login and fixed it before Ali saw it.</i>	Caught before you did	the agent patching a login bug under a green check
14	scene	<i>The same trick works for mobile apps and design tools too.</i>	Works beyond web	the agent checking a phone screen and a design canvas
15	info	<i>No front end? Let it verify the functions in code directly.</i>	Backend? Test functions	[info] note card — "no UI → check functions directly"
16	ali	<i>Ali saved this whole workflow as a reusable testing skill.</i>	Save it as a skill	Ali dropping the loop into a "Skills" drawer [overlay] "e2e-chrome-testing"
17	ali	<i>How far could a self-testing agent go before it needs you?</i>	Your turn	Ali turning to the viewer, thoughtful

SC-4 — Define test cases in BDD format [compulsory]

Concept: A plan is too vague to test against, so write each feature as concrete Given/When/Then behavior cases, organized in a folder and reviewed before running.

Ali's scenario: Ali's plan says "users can invite teammates" and the agent can't verify it, so he rewrites it as precise BDD cases.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Turn each feature into a precise Given, When, Then test case.</i>	Write BDD cases	Ali writing a three-line test card
2	scene	<i>Ali's plan said 'users can invite teammates' and little more.</i>	A vague plan	Ali frowning at a one-line plan item

3	ali	<i>The agent could not verify something so fuzzy.</i>	Too vague to test	the agent shrugging at a blurry requirement
4	scene	<i>Vague requirements trip up everyone, and that is normal.</i>	We all write vague first	warm scene, Ali nodding
5	info	<i>BDD writes behavior as Given, When, Then.</i>	What is BDD?	[info] define BDD — a 3-row Given/When/Then template
6	info	<i>Given an admin, when they invite an email, then an invite is sent.</i>	A real BDD case	[info] a filled Given/When/Then card using the invite example
7	info	<i>And then the invited user appears in the members list.</i>	Verifiable outcomes	[info] the card gains a checkable second "then" line
8	scene	<i>This precision feels tedious, yet it makes testing possible.</i>	Precision pays off	warm scene, Ali steady and focused
9	info	<i>Keep the cases in a folder, like docs slash features.</i>	Organize the files	[info] a folder tree — docs/features/invite.md
10	ali	<i>Ali writes cases one feature at a time, not all at once.</i>	One feature at a time	Ali finishing one tidy file before starting the next
11	ali	<i>He reviews and edits each case before letting it run.</i>	Review before running	Ali editing a case with a pen
12	info	<i>No plan? The agent can read the code to draft the cases.</i>	No plan, read the code	[info] note card — "codebase → test cases"
13	ali	<i>He runs them, compares results, and refines the gaps.</i>	Run, compare, refine	Ali comparing two run results and marking a gap
14	info	<i>Precise cases give the agent something exact to verify against.</i>	Exact criteria	[info] contrast — a vague plan vs a sharp BDD case
15	ali	<i>Now Ali's features arrive with clear, checkable behavior.</i>	Checkable behavior	Ali confident beside a feature with a passing case
16	ali	<i>Which feature could you rewrite as Given, When, Then today?</i>	Your turn	Ali turning to face the viewer

Section B — Mental Models

SLO 1.9 — Identify and resist "tab tax" [advanced]

Concept: Do one problem at a time; context-switching charges a hidden cost ("tab tax") that quietly degrades both quality and speed.

Ali's scenario: Ali directs an agent to fix a bug while juggling fourteen tabs, finishes nothing by noon, then closes down to a single task and the work finally flows.

#	Mode	Voiceover (spoken)	Caption	Visual
1	info	<i>One problem at a time protects your speed and your quality.</i>	One task at a time	[info] focus-card — one bright task node beside many faint blurred ones
2	scene	<i>Ali is an engineer who directs AI agents to fix code.</i>	Meet Ali	Ali at a tidy desk in a calm ed-tech office
3	scene	<i>This morning, fourteen browser tabs glow across his screen.</i>	Fourteen tabs	Ali dwarfed by a wide wall of glowing tabs

4	ali	<i>He starts one bug fix, then jumps to read email.</i>	A quick jump	Ali's chair swiveling toward a new tab [overlay] the bug-fix tab dimming behind him
5	scene	<i>If that sounds like your day, you are far from alone.</i>	You're not alone	warm scene, several engineers with crowded screens
6	info	<i>Every switch quietly charges a fee called the tab tax.</i>	The tab tax	[info] coin-drop — a coin falls each time an arrow hops between tabs
7	info	<i>Refocusing after a jump can cost many minutes each time.</i>	The refocus cost	[info] bar chart — a tiny "jump" bar beside a tall "recovery" bar
8	ali	<i>By noon, Ali has touched ten tasks and finished none.</i>	Ten started, zero done	Ali amid half-open work, drained [overlay] ten progress bars stuck near zero
9	scene	<i>He feels busy, yet the real work sits untouched.</i>	Busy, not done	Ali staring at an unmoved bug ticket
10	scene	<i>Here is the calmer part, and you can start it today.</i>	A calmer way	soft light shift, the desk relaxing around Ali
11	ali	<i>Ali closes every tab except the one bug he chose.</i>	Close the rest	tabs vanishing until one remains [overlay] single bug-fix tab centered
12	ali	<i>He gives the agent one clear task and watches it work.</i>	One clear task	Ali calmly reviewing the agent's single diff [overlay] one prompt, one diff
13	info	<i>One finished task now beats ten half-started ones.</i>	Finish beats start	[info] comparison — one full progress bar vs ten stuck bars
14	ali	<i>The fix lands, tests pass, and his mind feels quiet.</i>	Done and calm	green check on screen, Ali exhaling
15	info	<i>Batch the small stuff; give deep work a clean lane.</i>	One lane for depth	[info] two-lane diagram — a "quick tasks" lane and a protected "deep work" lane
16	ali	<i>How many tabs are open right now, and which one truly matters?</i>	Your turn	Ali turning gently toward the viewer

SLO 1.10 — Apply "friction maxxng" to engineer focus [intermediate]

Concept: Friction maxxng = deliberately ADD inconvenience to low-value distractions while REMOVING barriers to deep work, using environmental design instead of finite willpower.

Ali's scenario: Ali keeps losing agent sessions to tabs and his phone, so instead of trying harder he re-engineers his surroundings.

#	Mode	Voiceover (spoken)	Caption	Visual
1	info	<i>Design your environment so focus is the easy path.</i>	Design, don't fight	[info] two-path diagram — a downhill road to "deep work", uphill to "distraction"
2	scene	<i>Ali still loses hours to tempting tabs and his phone.</i>	The pull returns	Ali reaching for his phone mid agent-session
3	ali	<i>He tries harder to resist, but his willpower keeps running out.</i>	Willpower runs low	Ali white-knuckling focus while an energy meter drains
4	info	<i>Willpower fades through the day for every one of us.</i>	It's not a flaw	[info] line graph — an energy bar sloping down morning to evening
5	info	<i>So Ali tries friction maxxng: engineering barriers</i>	Friction maxxng	[info] definition card — "add friction to

		<i>around his attention.</i>		distractions, remove it from deep work"
6	info	<i>The idea: make good paths downhill and bad paths uphill.</i>	Downhill vs uphill	[info] hill diagram — easy slope to work, steep slope to distraction
7	ali	<i>First, Ali deletes the apps that keep pulling him away.</i>	Delete the bait	app icons dragged to the trash [overlay] social apps removed
8	ali	<i>Next, a website blocker guards every agent session.</i>	Block by default	a shield settling over the browser during a session [overlay] blocklist active
9	info	<i>A 2022 study found a phone in another room cut screen time thirty percent.</i>	Phone away = -30%	[info] stat callout — phone moved to the next room, a 30% down arrow
10	ali	<i>So Ali leaves his phone in the next room, face down.</i>	Phone, next room	Ali walking the phone out, a door between him and it
11	info	<i>He works in focused sprints using a Pomodoro timer.</i>	Work in sprints	[info] timer — a 25-minute block then a short break
12	ali	<i>His laptop's downtime mode silences every ping during deep work.</i>	Silence the pings	an OS "downtime" toggle switching on [overlay] Do Not Disturb on
13	scene	<i>He even builds one clean corner meant only for agent work.</i>	A dedicated space	minimal desk, no clutter, single screen, calm light
14	scene	<i>None of this needs more discipline, only a kinder setup.</i>	Setup over grit	Ali relaxed while the environment does the work
15	info	<i>This borrows from BJ Fogg: behavior follows how easy things are.</i>	Rooted in behavior design	[info] note card — "make the wanted behavior the easiest one"
16	info	<i>Willpower is finite, but a good environment keeps your commitments automatically.</i>	Environment keeps promises	[info] contrast bars — a draining willpower bar vs a steady environment bar
17	ali	<i>Now the deep-work path is the easiest one Ali can take.</i>	Easy to focus	Ali gliding down the gentle slope toward his agent work
18	ali	<i>Which single barrier could you add today to protect your focus?</i>	Your turn	Ali turning to the viewer

SLO 1.11 — Maintain dedicated time blocks for agent work [advanced]

Concept: Protect deep-work time by block-scheduling your calendar for agent work, so small daily wins compound instead of being nibbled away.

Ali's scenario: Ali's clean environment still can't help because his calendar is shredded into scraps, so he books protected ninety-minute blocks and defends them.

#	Mode	Voiceover (spoken)	Caption	Visual
1	info	<i>Block time on your calendar so agent work actually happens.</i>	Block the time	[info] calendar with one bold, protected block highlighted
2	scene	<i>Ali's environment is clean, yet his calendar is pure chaos.</i>	A new leak	Ali frowning at a calendar packed with tiny meetings
3	ali	<i>Meetings and pings nibble his day into useless scraps.</i>	Nibbled away	a calendar day eaten into fragments [overlay] 15-minute gaps scattered
4	scene	<i>Agent work needs long runs, but only crumbs</i>	No room to think	Ali eyeing the tiny gaps, unable to start

		<i>remain.</i>		
5	scene	<i>A fragmented calendar happens to almost everyone who does deep work.</i>	You're not alone	several calendars side by side, all fragmented
6	info	<i>Time blocking means reserving whole chunks for one kind of work.</i>	Time blocking	[info] calendar showing a solid 90-minute "agent work" block
7	info	<i>Treat that block like an appointment you never cancel on yourself.</i>	Keep the appointment	[info] the block styled as a locked calendar invite
8	ali	<i>Ali books two ninety-minute agent blocks before anything else.</i>	Book it first	Ali dragging two bold blocks onto the week [overlay] two 90-minute blocks
9	ali	<i>He marks them busy so no meeting can sneak in.</i>	Marked busy	the blocks shaded, meeting requests bouncing off [overlay] a "busy" shield
10	scene	<i>Protecting time is not selfish; it is how good work ships.</i>	Not selfish	Ali calm as colleagues respect the block
11	ali	<i>Inside a block, Ali runs one agent task start to finish.</i>	One clean run	Ali fully absorbed while the agent works end to end
12	info	<i>Uninterrupted blocks let small daily wins stack up over time.</i>	Wins that stack	[info] bars stacking taller across a week
13	info	<i>That stacking is compounding: today's progress lifts tomorrow's.</i>	Compounding	[info] a rising curve where each day builds on the last
14	ali	<i>Week by week, Ali's blocked hours turn into shipped features.</i>	Blocks become results	calendar blocks transforming into completed features [overlay] a shipped checklist growing
15	ali	<i>The same hours, once scattered, now compound into real momentum.</i>	Momentum	Ali confident, steady upward motion behind him
16	ali	<i>Where on this week's calendar could you fence off your first block?</i>	Your turn	Ali turning to the viewer beside an open calendar

SLO 1.12 — Map vs. Territory: read what the agent did [advanced]

Concept: When a prompt fails, study what the agent actually produced instead of just rewording your request — the map is not the territory.

Ali's scenario: Ali asks his agent to add caching to the courses API; it stays slow, and he keeps rewording instead of reading what the agent really built.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>When a prompt fails, look at what the agent actually built.</i>	Read reality first	Ali at his desk pointing calmly at a code screen
2	scene	<i>Ali asked his agent to add caching to the courses API.</i>	The request	Ali typing, a glowing API diagram beside him
3	scene	<i>The agent returned code, but the endpoint stayed just as slow.</i>	Still slow	a spinning loader, Ali frowning slightly
4	ali	<i>His first instinct was to reword the prompt and rerun it.</i>	The reword reflex	Ali retyping into a chat bubble, arrows looping
5	info	<i>This is editing your map, not exploring the real territory.</i>	Map vs. territory	[info] split-panel — a neat drawn map on the left, messy real land on the right

6	scene	Everyone reaches for the reword button first; it feels productive.	You're not alone	warm scene, Ali with a small shrug and half-smile
7	ali	So Ali slowed down and actually read the agent's output.	Read the output	Ali leaning in, eyes on the code
8	ali	The agent had cached the wrong function entirely.	Wrong function cached	red highlight on a mislabeled function [overlay] "cached: getUserPrefs()" vs intended "getCourses()"
9	scene	His words were fine; his picture of the code was off.	The real gap	Ali's expression shifting to realization
10	info	The map is your idea; the territory is the running code.	Two different things	[info] labeled diagram — "MAP = your mental model" over "TERRITORY = actual output"
11	ali	He revised his mental model, then wrote a precise prompt.	Revise, then rewrite	Ali updating a sticky-note model on the wall [overlay] sticky note edited to "cache getCourses response"
12	scene	This time the agent cached the exact endpoint he meant.	It worked	green check, a fast response bar
13	scene	Slow down here, because reading beats rewording almost every time.	Reading wins	warm, Ali nodding gently
14	info	Three moves: read the output, understand its interpretation, revise your model.	The loop	[info] a 3-step list that builds up one line at a time
15	ali	Now Ali treats every failure as a clue about reality.	Failures are clues	Ali holding a magnifying glass over code
16	ali	When did you last reword a prompt instead of reading the result?	Your turn	Ali turning to face the viewer

SLO 1.13 — Hollow vs. solid AI vocabulary [advanced]

Concept: Audit whether the AI terms you use daily point to things you genuinely understand, using the Explain-It-Like-I'm-5 test — hollow words crumble under one real question.

Ali's scenario: Ali confidently says "embedding" in standup, then a new teammate asks what it means and he freezes.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	Test whether your AI words point to things you truly understand.	Solid vs. hollow	Ali holding two word-cards, weighing them
2	scene	In standup, Ali said the agent needed better embeddings.	The standup	Ali at a whiteboard, team listening
3	scene	A new teammate asked what an embedding actually is.	The question	a junior engineer raising a hand
4	ali	Ali opened his mouth and nothing solid came out.	The freeze	Ali blank-faced, a small sweat drop
5	scene	That silence happens to almost everyone; it's completely normal.	It's normal	warm scene, the whole team nodding with him
6	info	Hollow words sound right but crumble under one real question.	The crumble test	[info] a word that looks solid, then cracks into pieces
7	ali	He had repeated 'embedding' for months without a real referent.	No referent	the word floating, unanchored [overlay] "embedding" with a dangling,

				unconnected string
8	info	<i>Solid understanding traces a word back to something concrete.</i>	Trace it down	[info] a word connected by a line to a real object
9	ali	<i>So Ali tried the Explain-It-Like-I'm-5 test.</i>	The ELI5 test	Ali talking to a small child figure
10	ali	<i>An embedding turns meaning into numbers a computer can compare.</i>	Meaning into numbers	a word transforming into coordinates on a grid [overlay] "cat → [0.2, 0.9, 0.1 ...]"
11	scene	<i>Saying it plainly showed him which parts he actually knew.</i>	Plain talk reveals gaps	Ali with a half-lit lightbulb
12	info	<i>Audit five terms: context window, fine-tuning, hallucination, RAG, embedding.</i>	Five to check	[info] a checklist of five AI terms, empty boxes
13	scene	<i>If some crumble, good; that's exactly where to learn next.</i>	Gaps are progress	warm, Ali circling one term with a smile
14	ali	<i>Ali now explains each term to an imaginary five-year-old.</i>	Practice plainly	Ali rehearsing aloud, gesturing
15	ali	<i>Words he can put plainly are words he genuinely owns.</i>	Plain equals owned	Ali confident, a word firmly anchored down
16	ali	<i>Which AI word could you not yet explain to a child?</i>	Your turn	Ali facing the viewer

SLO 1.14 — Fresh problem-solving vs. rote prompt recycling [advanced]

Concept: Catch yourself mechanically recycling prompt templates and applying frameworks without thinking — that's cargo culting — and instead reason about each problem from first principles.

Ali's scenario: Ali has a "magic" prompt template he pastes into every task, until a migration task where it silently produces wrong results.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Solve each problem fresh instead of recycling the same prompt.</i>	Think, don't recycle	Ali choosing between a saved template and a blank page
2	scene	<i>Ali kept a magic prompt template that once worked beautifully.</i>	The magic template	a glowing saved snippet in a frame
3	ali	<i>He pasted it into every new task out of pure habit.</i>	Paste and pray	Ali copy-pasting the same block repeatedly [overlay] identical prompt pasted across three task tabs
4	scene	<i>For a new migration task, the output looked fine but wasn't.</i>	Quietly wrong	a green check with a hidden bug underneath
5	info	<i>This is cargo culting: doing the ritual, missing the mechanism.</i>	Cargo culting	[info] define-the-term panel — a ritual icon beside a real gears icon
6	scene	<i>We all collect prompts that once felt like little spells.</i>	We all do it	warm scene, Ali holding a small spellbook, smiling
7	ali	<i>The template assumed a context this task did not have.</i>	Wrong assumptions	a puzzle piece that doesn't fit the slot [overlay] template "assumes: fresh DB" vs task "existing data"
8	ali	<i>Ali had followed the tutorial steps without asking why.</i>	Steps without the why	Ali on autopilot, moving robotically

9	info	<i>Fresh thinking starts from the problem, not the template.</i>	Start from the problem	[info] an arrow from "the problem" to a tailored prompt
10	ali	<i>So he asked what this specific task truly required.</i>	What does THIS need	Ali pausing, thinking from first principles
11	ali	<i>He rebuilt the prompt from the migration's real constraints.</i>	Build from constraints	Ali assembling a prompt from parts [overlay] constraint chips "existing rows", "rollback safe", "idempotent"
12	scene	<i>The new prompt caught the edge case the template had missed.</i>	It caught the bug	a bug trapped in a net, Ali relieved
13	scene	<i>Reusing prompts isn't wrong; reusing them blindly is the trap.</i>	Blind reuse is the trap	warm scene, a template with an eye opening on it
14	info	<i>Ask each time: does this template's assumption fit this problem?</i>	The checkpoint	[info] a yes/no decision node with two branches
15	ali	<i>Now Ali treats templates as starting points, never as autopilot.</i>	Starting point, not autopilot	Ali steering, both hands on the wheel
16	ali	<i>Where are you running a prompt ritual without knowing why?</i>	Your turn	Ali facing the viewer

SLO 5.2 — Debug with the scout mindset [optional]

Concept: When an agent's output is wrong, the "soldier" defends the prompt while the "scout" investigates with curiosity — truth-seeking beats position-defending.

Ali's scenario: Ali's coding agent breaks the build; he first blames the agent, then switches to investigating what it actually understood.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>When an agent fails, investigate like a scout, not a soldier.</i>	Scout, not soldier	Ali standing between a soldier figure and a scout figure
2	ali	<i>Ali's coding agent shipped a function that broke the tests.</i>	Tests failing	Ali staring at a wall of red failing tests
3	ali	<i>His gut said the agent was the problem.</i>	"The agent's broken"	Ali pointing an accusing finger at the screen
4	scene	<i>That defensive reflex is human, and every engineer feels it.</i>	The reflex is normal	Ali sighing, a warm mentor hand on his shoulder
5	scene	<i>A soldier defends the prompt; a scout hunts for the truth.</i>	Two mindsets	a soldier with a shield beside a scout with a spyglass and map
6	info	<i>Two mindsets ask two very different questions.</i>	Two questions	[info] two-column compare — Soldier "It should work" / Scout "What happened?"
7	ali	<i>So Ali paused and asked what the agent actually understood.</i>	What did it understand?	Ali leaning in, curious, reading the agent's reasoning
8	ali	<i>He reread his prompt as if he were the agent.</i>	Read it as the agent	Ali wearing agent-goggles, prompt text floating [overlay] the prompt as the agent sees it
9	ali	<i>One line said 'update the record' without saying which one.</i>	Ambiguous instruction	highlighted vague line "update the record"

10	info	<i>The scout traces a failure back to its source.</i>	Trace it back	[info] flow diagram — prompt → agent reading → wrong action → broken test, looping back
11	scene	<i>Finding your own gap here is a win, not a shame.</i>	A gap found is a win	Ali with a small smile, a soft lightbulb
12	ali	<i>The soldier would have rerun the prompt and failed again.</i>	Rerunning won't fix it	Ali trapped in a loop arrow, same error repeating
13	ali	<i>Ali named the exact record and ran it again.</i>	Precise instruction	prompt now reads "update the student record by ID"
14	ali	<i>The agent got it right, and the tests turned green.</i>	Tests passing	green checkmarks cascading, Ali relieved
15	info	<i>Curiosity found the bug that defensiveness would hide.</i>	Curiosity wins	[info] takeaway card — "Truth-seeking beats position-defending"
16	scene	<i>Next agent failure, ask: what did it truly understand?</i>	Your turn	Ali holding a spyglass out toward the viewer

SLO 5.1a — Connect agent work to real stakes [advanced]

Concept: Move from dry, abstract engagement to visceral, concrete stakes — when you feel who the work is for, you catch errors the disengaged mind skips.

Ali's scenario: Ali treats a retention-data analysis as dry numbers, drifts, then reframes it as 50,000 children's learning and catches a hidden error.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>Feel the real stakes, and your agent's work gets sharper.</i>	Feel the stakes	Ali at his desk, a warm classroom glowing behind him
2	ali	<i>Ali asked his agent to analyze student retention data.</i>	Analyze retention	Ali facing a dashboard of numbers
3	ali	<i>To him it was rows, columns, and dry percentages.</i>	Only numbers	a gray spreadsheet, Ali's eyes glazing over
4	info	<i>Abstract framing: 'AI can analyze retention data.'</i>	Flat and abstract	[info] a card showing a flat, gray abstract statement
5	scene	<i>Drifting on abstract data is normal, and not a failing.</i>	It's normal to drift	Ali gently yawning, soft non-judgmental palette
6	scene	<i>A number on a screen is not a face in a classroom.</i>	Number vs face	split — a gray "12%" beside a child's face in class
7	ali	<i>Then Ali remembered whose learning these numbers were.</i>	Whose learning?	Ali looking up, a classroom forming in his mind
8	info	<i>This analysis shapes materials for 50,000 children in Balochistan.</i>	Real stakes	[info] a map of Balochistan filling with small child icons
9	ali	<i>Suddenly the dry percentages had faces and futures.</i>	Numbers gained faces	spreadsheet cells morphing into tiny portraits
10	scene	<i>Caring more here helps you; it never slows you down.</i>	Care sharpens you	Ali energized, warm light spreading across his desk
11	ali	<i>Reengaged, Ali spotted a filter that dropped rural students.</i>	A hidden filter	Ali catching a highlighted line excluding a whole region

12	ali	<i>The disengaged version of him would have missed it.</i>	The drift would miss it	a ghosted Ali scrolling straight past the error
13	info	<i>Stakes you feel pull your attention to what matters.</i>	Care → focus	[info] attention diagram — care → focus → caught errors
14	ali	<i>He fixed the filter, and every child was counted.</i>	Everyone counted	the full map, all icons included, Ali satisfied
15	info	<i>Thinking that grips your care beats thinking that floats.</i>	Gripped beats floating	[info] takeaway card — gripped vs floating notation
16	scene	<i>Before your next task, ask: who is this really for?</i>	Your turn	Ali gesturing from the classroom toward the viewer

SLO 5.4 — Shift from specialist to AI orchestrator [compulsory]

Concept: The biggest barrier to AI adoption is identity, not skill — reframing "I am a coder" into "I am a system architect who orchestrates AI" unlocks a more interesting role.

Ali's scenario: Ali, who gets his dopamine from clean code, feels deskilled managing agents, then reframes himself as a conductor of a team of agents.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>The hardest part of using AI agents is identity, not tech.</i>	It's identity, not tech	Ali facing a mirror whose reflection is shifting
2	ali	<i>Ali loved the quiet joy of writing clean code.</i>	The joy of clean code	Ali smiling at an elegant block of code
3	ali	<i>Now his agent writes the code, and he reviews it.</i>	The agent codes now	an agent typing while Ali watches, slightly uneasy
4	ali	<i>He felt oddly deskilled, like his craft was slipping away.</i>	Feeling deskilled	Ali's tools fading from his hands
5	scene	<i>That loss you feel is real, and nearly everyone feels it.</i>	The loss is real	a mentor beside Ali, empathetic
6	info	<i>The barrier to AI adoption is mostly emotional, not technical.</i>	Mostly emotional	[info] bar chart — barrier: 20% technical / 80% identity
7	scene	<i>A soloist plays one instrument; a conductor leads a whole orchestra.</i>	Soloist to conductor	split — a lone violinist beside a conductor before an orchestra
8	ali	<i>Ali stopped asking, 'Am I still a coder?'</i>	The wrong question	Ali crossing out the "coder" label on his badge
9	ali	<i>He started asking, 'What system am I building?'</i>	The better question	Ali sketching an architecture diagram
10	ali	<i>His new role: an architect orchestrating a team of agents.</i>	System architect	Ali at the center with several agents around him
11	scene	<i>This role is more interesting than before, not less.</i>	More interesting, not less	Ali energized before a bigger canvas
12	info	<i>The coder wrote lines; the orchestrator designs the whole flow.</i>	A level up	[info] compare — before "writing functions" / after "designing systems + directing agents"
13	ali	<i>Clinging to only coding would shrink Ali's impact.</i>	Clinging shrinks you	Ali cramped in a tiny box, agents sitting idle

14	ali	<i>Conducting well, he now ships what a team once shipped.</i>	A team's output	Ali directing agents producing many features at once
15	info	<i>Your craft moves up a level; it does not disappear.</i>	Craft moves up	[info] ladder diagram — rung 1 "write code" → rung 2 "orchestrate systems"
16	ali	<i>Ali still feels the craft, now at the scale of systems.</i>	Craft, at system scale	Ali content, conducting a smooth pipeline
17	scene	<i>Ask yourself: what system do you want to conduct next?</i>	Your turn	Ali offering the baton toward the viewer

SLO 5.3 — Map nature's patterns onto software (biomimetics) [optional]

Concept: Nature has optimized systems for 3.8 billion years, so many algorithms are direct translations of biology — the transferable habit is to ask "how has nature already solved this?"

Ali's scenario: Ali is stuck balancing load across busy servers, then borrows ant-trail and honeybee patterns from nature to route traffic evenly.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>When stuck, ask how nature already solved your problem.</i>	Ask nature first	Ali at his desk, a leaf and a circuit blending together
2	ali	<i>Ali needed to route tasks across many busy servers.</i>	Routing across servers	Ali facing a tangle of server nodes
3	ali	<i>Every design he tried overloaded one machine or another.</i>	Servers overloading	one node glowing red while others sit idle
4	scene	<i>Feeling stuck on a hard systems problem is completely normal.</i>	Being stuck is normal	Ali with head in hands, a gentle mentor nearby
5	scene	<i>Nature is a research lab that has run for 3.8 billion years.</i>	Nature's old lab	an Earth timeline with life forms drawn as scientists
6	info	<i>Biomimetics means copying nature's solutions into engineering.</i>	Biomimetics	[info] definition card — "Biomimetics = bio (life) + mimesis (imitate)"
7	ali	<i>So Ali asked, how do ants find the shortest path?</i>	How do ants do it?	Ali watching an ant trail cross his notebook
8	info	<i>Ants leave trails, and the best routes attract the most followers.</i>	Ant trails	[info] ant-colony diagram — pheromone trails strengthening the shortest path
9	ali	<i>He gave his servers a digital scent for fast responses.</i>	A digital scent	server nodes marked with glowing trail markers
10	info	<i>Honeybees balance foragers by dancing about the richest flowers.</i>	Bee waggle dance	[info] waggle-dance diagram — bees redirecting toward the best sources
11	ali	<i>Ali borrowed that idea to shift load toward free machines.</i>	Balancing like bees	tasks flowing from busy nodes to free ones
12	scene	<i>You need not invent from scratch; nature shares its patents.</i>	Nature shares its work	Ali relieved, an open book of nature's designs
13	ali	<i>Fighting the problem alone kept Ali circling the same walls.</i>	Going it alone stalls	a ghosted Ali looping with no progress
14	ali	<i>His routing balanced smoothly, inspired by ants and bees.</i>	Balanced traffic	an even glow across all servers, Ali pleased

15	info	Immune systems, slime molds, and swarms hold more answers.	More patterns	[info] gallery card — immune defense, slime-mold networks, flocking swarms
16	scene	Next hard problem, ask: how has nature solved this already?	Your turn	Ali gazing out a window full of nature

Section C — Data Mastery

SLO 5.1b — Why we need evals (types, BDD, LLM-as-a-Judge) [compulsory]

Concept: Evals are the measuring stick for whether an agent's output is actually good.

Ali's scenario: Ali's coding agent ships a feature that looks fine, but he has no real way to know if it's good.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	Evals are the measuring stick for whether your agent's work is good.	Evals = the measuring stick	Ali holding a tape measure up to a glowing code panel
2	ali	Ali's coding agent shipped a feature, and it looked fine at first.	Looks fine at first	Ali nodding at a screen of green checkmarks
3	scene	If you have ever trusted a quick glance here, you are in good company.	We have all been there	warm scene, Ali hand on chin
4	ali	But a glance told Ali nothing about whether it was actually right.	A glance proves nothing	a magnifying glass revealing a hidden bug
5	ali	He learned a simple truth: you cannot improve what you cannot measure.	Measure, then improve	a broken ruler swapped for a clean one
6	info	An eval defines what a good output looks like, then scores the agent against it.	What is an eval?	[info] definition card — "EVAL = criteria + score"
7	info	There are three kinds of evals Ali leans on most.	Three kinds of evals	[info] a list template reveals 3 empty rows
8	info	Code-based checks test exact, measurable things automatically.	1. Code-based checks	[info] row 1 fills — "Exact / automatic"
9	info	Human review catches the judgment calls a script would miss.	2. Human review	[info] row 2 fills — "Nuance / judgment"
10	info	And an LLM-as-a-Judge uses a model to score outputs at scale.	3. LLM as a Judge	[info] row 3 fills — "Model scores model"
11	info	Ali writes each expectation as Given, When, Then — a style called BDD.	BDD: Given / When / Then	[info] define BDD, a 3-row template appears
12	ali	Given a bug report, when the agent fixes it, then the tests pass.	A behavior spec in action	Ali reading a three-line spec card [overlay] "Given / When / Then"
13	ali	For fuzzy work, Ali asks another model to judge the output.	LLM as a Judge	Ali handing an essay to a robot holding a scorecard
14	info	The judge model scores each answer against clear, written criteria.	Score against criteria	[info] rubric card — Accuracy 4/5, Clarity 5/5
15	scene	This part feels strange at first, and that is completely okay.	Strange but powerful	warm scene, Ali smiling, reassured
16	ali	Now Ali sees quality as a number he can watch improve.	Quality you can watch	Ali watching a score line rise on a chart

17	ali	Ask yourself: how would you prove your agent's last output was good?	Your turn	Ali turning to face the viewer
----	-----	--	-----------	--------------------------------

SLO 5.1c — Critically evaluate an agent's database decisions [advanced]

Concept: Don't trust blindly — the agent proposes database changes, but you review and question them before they touch real data.

Ali's scenario: Ali's agent wants to reorganize and migrate the live learner-records database.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	Never let your agent change a real database before you review its plan.	Review before it runs	Ali standing between an eager robot and a locked database vault
2	ali	Ali's agent offered to reorganize the entire learner-records database.	A bold proposal	the robot presenting a blueprint of tables to Ali
3	scene	When the plan sounds confident, saying yes feels easy.	Confidence is tempting	a big glowing "Approve" button, Ali's finger hovering
4	ali	But confident and correct are not the same thing.	Confident is not correct	two speech bubbles, one polished, one with a hidden crack
5	ali	So Ali treats the agent like a junior teammate handing him a pull request.	Propose, review, approve	Ali reviewing a document from the robot [overlay] "Propose → Review → Approve"
6	info	He checks three things before anything runs.	Three checkpoints	[info] a list template, 3 empty rows
7	info	First, the schema: does the table design actually fit the data?	1. Schema	[info] row 1 — "Right shape?"
8	info	Second, the queries: will they run fast, or crawl?	2. Queries	[info] row 2 — "Fast or slow?"
9	info	Third, migrations: can this change be undone if it breaks?	3. Migrations	[info] row 3 — "Reversible?"
10	ali	Reading closely, Ali sees the agent dropping a column instead of renaming it.	A costly mistake, caught	Ali circling a red line; a column icon crumbling
11	scene	Catching this can feel slow, and that patience is worth it.	Slow now saves hours	warm scene, Ali calm beside an hourglass
12	ali	He asks the agent to explain its reasoning before approving.	Ask it to explain	Ali with a question mark, the robot showing its notes
13	ali	Then he tests the migration on a copy, never on live data.	Test on a copy first	two databases — a glowing "copy" and a shielded "live" one
14	ali	The real learner data stays safe, and Ali stays in control.	Safe data, real control	Ali locking the vault with a confident smile
15	ali	Ask yourself: what would you check before letting an agent edit real data?	Your turn	Ali turning to the viewer

SLO 5.1d — Relational vs. flat data [advanced]

Concept: Move beyond "one big sheet" (flat) to a relational model — tables linked by keys — because a spreadsheet breaks down as data and relationships grow.

Ali's scenario: Ali tracks course enrollments in one giant spreadsheet that keeps breaking.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>Relational data splits one giant sheet into smaller tables linked by keys.</i>	Split, then link	one bloated spreadsheet splitting into three neat linked cards
2	ali	<i>Ali tracked every course enrollment in a single, enormous spreadsheet.</i>	One sheet to rule them all	Ali dwarfed by a spreadsheet stretching off-screen
3	scene	<i>Starting with one big sheet feels natural, and most of us do.</i>	We all start here	warm scene, Ali cheerfully typing into a grid
4	ali	<i>But every student's name was retyped on every single row.</i>	The same data, over and over	a name highlighted, repeated down many rows
5	info	<i>That is flat data: one wide sheet, with everything repeated.</i>	Flat data, defined	[info] definition card — "Flat = one sheet, repeated data"
6	ali	<i>When one student changed her name, Ali had to fix fifty rows.</i>	One change, fifty edits	Ali sweating, red marks down a sheet
7	scene	<i>And a single missed typo quietly broke all his totals.</i>	One typo, broken totals	a chart tilting, a small red error glinting
8	info	<i>Relational data means separate tables linked by a shared key.</i>	Relational, defined	[info] definition card — "Relational = linked tables"
9	info	<i>Ali splits it into Students, Courses, and Enrollments.</i>	Three focused tables	[info] three linked table cards appear
10	info	<i>A key is a shared ID that links one table to another.</i>	Keys do the linking	[info] an arrow from student_id in one table to another
11	ali	<i>Now each student's name lives in exactly one place.</i>	One name, one home	Ali relaxed, a single glowing student card
12	scene	<i>This shift takes a moment to click, and that is completely normal.</i>	Give it a moment	warm scene, a lightbulb softly warming over Ali
13	info	<i>As data grows, Ali adds a table, not another hundred columns.</i>	Scales by tables	[info] comparison — a flat sheet stretching vs tidy tables added
14	ali	<i>He changes a name once, and every course updates instantly.</i>	Change once, update everywhere	Ali edits one card; ripples update the linked tables
15	ali	<i>Ask yourself: where in your data are you repeating the same thing?</i>	Your turn	Ali turning to the viewer

SLO 5.1e — Iterate on analysis (3+ cycles) [advanced]

Concept: Good data analysis is refined through iteration — the first pass is a draft, and each cycle sharpens the query, cleans the visualization, and deepens the insight.

Ali's scenario: Ali investigates why students drop off in week three; his first chart misleads, so he iterates three times.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>Good data analysis is refined in cycles, never nailed in one shot.</i>	Analysis is iterative	Ali at an easel with three progressively cleaner charts
2	ali	<i>Ali wanted to know why students quit the course in week three.</i>	The question	Ali frowning at a drop-off in a line chart
3	scene	<i>The first answer is rarely the best one, and that is</i>	First pass is not	warm scene, a crumpled sketch beside

		<i>expected.</i>	best pass	a fresh page
4	ali	<i>His first query lumped every student into one blurry average.</i>	Cycle 1: the rough draft	a fuzzy, featureless bar looming over Ali
5	info	<i>Cycle one is a draft that raises more questions than it answers.</i>	A draft, not an answer	[info] cycle tracker — step 1 of 3 highlighted, "Draft"
6	ali	<i>So he sharpened the query to split students by experience level.</i>	Cycle 2: sharpen the query	Ali splitting one bar into several distinct ones
7	scene	<i>Refining can feel like backtracking, but it is real progress.</i>	Refining is progress	warm scene, Ali climbing steps made of chart bars
8	info	<i>With a cleaner query, a clearer pattern starts to appear.</i>	A pattern emerges	[info] cycle tracker — step 2 of 3, "Sharpen"
9	ali	<i>The chart was now correct, but cluttered and hard to read.</i>	Correct, but messy	Ali squinting at an overcrowded chart
10	ali	<i>So he cleaned the visualization down to one clear line.</i>	Cycle 3: clean it up	a tangled chart smoothing into a single crisp line
11	info	<i>Cycle three is a simple chart anyone can read at a glance.</i>	Clear at a glance	[info] cycle tracker — step 3 of 3, "Clarify"
12	ali	<i>Only now did the real insight surface: beginners left, experts stayed.</i>	The real insight	Ali's eyes widening at two clearly separated lines
13	info	<i>Three cycles carried him from draft, to sharpen, to clarify.</i>	Draft, sharpen, clarify	[info] the full 3-step tracker filled and glowing
14	ali	<i>Ali trusts this answer because he earned it through iteration.</i>	Earned, not guessed	Ali presenting the clean chart, confident
15	ali	<i>Ask yourself: what would your analysis reveal on a second or third pass?</i>	Your turn	Ali turning to the viewer

SLO 5.5 — Create a reusable data-analysis skill [optional]

Concept: Once an analysis pattern works, package it as a reusable "skill" — a saved, trigger-able procedure with steps and example output — so a one-off becomes a repeatable capability.

Ali's scenario: Ali keeps rebuilding the same enrollment analysis by hand every month, so he packages it as a skill.

#	Mode	Voiceover (spoken)	Caption	Visual
1	scene	<i>Package a working pattern once, and your agent can reuse it forever.</i>	Save once, reuse always	Ali dropping a glowing procedure into a labeled "Skills" drawer
2	ali	<i>Every month, Ali rebuilt the same enrollment report from scratch.</i>	The monthly grind	Ali wearily typing the same report, calendar pages flipping
3	scene	<i>Rebuilding familiar work by hand is a trap many of us fall into.</i>	A common trap	warm scene, Ali on a treadmill of repeating charts
4	ali	<i>He retyped the same steps and fixed the same small mistakes each time.</i>	Same work, again	Ali erasing and re-fixing an identical error
5	ali	<i>Then Ali realized the pattern itself could be saved.</i>	A better idea	a lightbulb over Ali, the report turning into a reusable card
6	info	<i>A skill is a saved procedure your agent can trigger on demand.</i>	What is a skill?	[info] definition card — "Skill = saved, trigger-able procedure"

7	info	<i>A good skill has three parts.</i>	Three parts	[info] a list template, 3 empty rows
8	info	<i>A trigger: the phrase or moment that calls the skill up.</i>	1. Trigger	[info] row 1 — "When to run"
9	info	<i>Steps: the exact sequence that does the work.</i>	2. Steps	[info] row 2 — "How to run"
10	info	<i>An example output: a sample of what 'done' looks like.</i>	3. Example output	[info] row 3 — "What good looks like"
11	ali	<i>Ali writes his enrollment analysis down as one reusable skill.</i>	Packaged once	Ali sealing his steps into a neat labeled card [overlay] "Enrollment Analysis skill"
12	scene	<i>It takes effort the first time, and it pays back quickly.</i>	Effort now, payback fast	warm scene, a small seed growing into a tree
13	ali	<i>Next month, a single trigger runs the whole analysis for him.</i>	One trigger, done	Ali clicking once; the full report assembling itself
14	ali	<i>A one-off task quietly became a repeatable capability.</i>	One-off to repeatable	Ali relaxed, a shelf of labeled skills behind him
15	ali	<i>Ask yourself: which task do you rebuild often enough to package once?</i>	Your turn	Ali turning to the viewer

Section D — Autonomous Operations

SLO 5.6 — Parallel work with git worktrees [optional]

Concept: A git worktree checks out several branches of one repo into separate folders at once, so multiple agents work the same codebase in parallel without stepping on each other.

Ali's scenario: Ali runs one agent on a bug fix and another on a big refactor; they keep clobbering the same files until he gives each its own worktree.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Git worktrees let Ali run two agents on one repo at once.</i>	The answer first	Ali at a desk, two glowing agent panels working side by side [overlay] "worktrees" tag over both
2	scene	<i>Ali starts two agents, and they clobber each other's files.</i>	The friction	two agent hands grabbing the same file, a spark of conflict
3	ali	<i>If parallel agents keep colliding, that frustration is completely normal.</i>	You're not alone	Ali exhaling, shoulders easing
4	info	<i>A worktree is a second working copy of the same repo.</i>	Define it	[info] definition card — one repo icon splitting into two labeled folder copies
5	ali	<i>Think of worktrees as separate desks in one shared workshop.</i>	The metaphor	Ali in a workshop with two tidy desks under one roof [overlay] "desk A / desk B"
6	info	<i>Each desk holds a different branch, isolated but sharing one history.</i>	Isolated, shared roots	[info] two-desk diagram — separate branches feeding one common history
7	ali	<i>Ali makes a worktree for the bug fix in its own folder.</i>	The fix, step one	Ali spinning up folder A [overlay] git worktree add ../fix hotfix
8	ali	<i>He makes a second worktree for the big refactor.</i>	The fix, step two	Ali spinning up folder B beside the first

				[overlay] git worktree add ../refactor cleanup
9	info	<i>The add command creates the folder and the branch together.</i>	One command, two things	[info] command-breakdown — arrow from command to "folder + branch"
10	ali	<i>Now each agent edits its own folder without touching the other.</i>	No more collisions	two agents working calmly in separate desks
11	ali	<i>Setup feels fiddly at first, and that is okay.</i>	Be patient here	Ali nodding, a reassuring gesture
12	scene	<i>One day Ali deletes a folder by hand and git gets confused.</i>	The failure mode	an empty desk with a red "?" where a worktree used to be
13	info	<i>Run git worktree prune to clear the stale entry.</i>	The recovery	[info] warning-then-fix — a red stale entry cleared by git worktree prune
14	ali	<i>Both agents finish, and Ali merges two clean branches.</i>	The payoff	Ali merging two branches into the trunk [overlay] "fix + refactor → main"
15	info	<i>Two tasks done in parallel, with zero merge collisions.</i>	The result	[info] before/after tally — "1 task, conflicts" vs "2 tasks, 0 conflicts"
16	ali	<i>Where in your work could two agents run side by side?</i>	Your turn	Ali turning to face the viewer

SLO 5.7 — Score agent output with rubrics [compulsory]

Concept: Build a rubric of clear criteria — completeness, accuracy, efficiency, safety, style — and score agent outputs against it, starting manually and progressing to automated evals.

Ali's scenario: Ali's agent ships code that "looks right" but sometimes breaks; he replaces his gut feeling with a five-criterion rubric, then automates the scoring.

#	Mode	Voiceover (spoken)	Caption	Visual
1	ali	<i>Ali rates every agent output against a clear rubric.</i>	The answer first	Ali holding a scorecard beside a code panel [overlay] five checkboxes on the card
2	scene	<i>Ali's agent ships code that looks right but sometimes breaks.</i>	The friction	polished-looking code with a hidden crack running through it
3	ali	<i>Trusting a gut feeling here is normal, and also risky.</i>	You're not alone	Ali with a thought bubble "looks fine?"
4	info	<i>Andrew Ng calls evaluation the biggest predictor of agent success.</i>	Why it matters	[info] quote card — a pull-quote on evaluation
5	info	<i>A rubric is a scorecard of criteria you grade output against.</i>	Define it	[info] definition card — a blank scorecard filling with criteria rows
6	ali	<i>It bridges Ali from 'that looks right' to real measurement.</i>	The metaphor	Ali crossing a bridge from a foggy "gut" bank to a marked "measurement" bank [overlay] "feeling → number"
7	info	<i>Ali's rubric has five criteria to check every time.</i>	The structure	[info] a 5-item list building one row at a time
8	info	<i>Completeness, accuracy, efficiency, safety, and style, each scored one to five.</i>	The criteria	[info] five criteria rows, each with a 1–5 scale
9	ali	<i>Ali starts by scoring ten outputs by hand.</i>	Start manual	Ali marking scores on ten cards [overlay] tally "10 outputs scored"

10	ali	<i>Suddenly the weak spots in his agent stand out clearly.</i>	Patterns appear	two low-scoring cards glowing red among the stack
11	ali	<i>Manual scoring feels slow, and that patience pays off.</i>	Be patient here	Ali steady and focused
12	info	<i>Once the rubric is stable, Ali automates the scoring.</i>	Progress to automated	[info] manual-to-auto arrow — a hand icon transitioning to a gear/eval icon
13	scene	<i>An automated score passes, but the code quietly ignored safety.</i>	The failure mode	a green "PASS" badge with a small red safety flag hidden behind it
14	info	<i>Ali weights safety highest so it can never be skipped.</i>	The fix	[info] weighted-rubric card — the safety row enlarged with a heavier weight
15	ali	<i>Now Ali measures every agent the same fair way.</i>	The payoff	Ali comparing several agents on one consistent scoreboard [overlay] "same rubric, every agent"
16	info	<i>Gut feeling became a repeatable number he can trust.</i>	The result	[info] before/after — a fuzzy "?" transforming into a steady score
17	ali	<i>Which five criteria would define good work for your agent?</i>	Your turn	Ali turning to the viewer beside a blank five-row scorecard